TEHNOLOGIJE RAČUNARSTVA U OBLAKU

Hardversko/softverska pozadina

dr Nebojša Horvat

Septembar 2026

SVRHA PREDAVANJA

- Pregled **osnovnih tehnologija** koje čine računarstvo u oblaku mogućim:
  - **Virtuelizacija**
  - **Kontejnerizacija**

VIRTUELIZACIJA

Zamišljeni a stvarni računari.

ŠTA JE VIRTUELIZACIJA

- **Virtuelizacija** – jedan fizički računar se ponaša kao **više nezavisnih, potpuno odvojenih** računarskih sistema
- Granica nije oštra – računari ovaj trik rade **decenijama**
  - i **procesi** su delimično odvojeni i ponašaju se kao da su “sami na svetu” (npr. memorija)
- Još je teže odvojiti virtuelizaciju od **emulacije**

EMULACIJA I VIRTUELIZACIJA

- Sistem koji se **pretvara da je drugi sistem** – to je u samim osnovama oblasti
- **Čerč-Tjuringova teza**: **svaki sistem može da oponaša svaki drugi** (performanse i prostor na stranu)
- U najgorem slučaju: **emulator** koji tačno oponaša svaki digitalni čip u sistemu
  - postoje takvi emulatori za neke **stare igračke mašine**

RAZLIKA IZMEĐU EMULACIJE I VIRTUELIZACIJE

- Razlika **nije u nameni ni efektu**, već u **tehnologiji i performansama**
- **Virtuelizacija** → performanse **uporedive sa pravim sistemom**
  - zauzvrat prihvatamo **ograničenja**
- **Emulacija** → može skoro sve, ali **sporo**
- Primer: PC može da *emulira* čip **6502**, ali ga nikad neće *virtuelizovati*

NAJOPŠTIJA DEFINICIJA VIRTUELIZACIJE

- Ovde se bavimo virtuelizacijom **računarskih sistema**, ali se gotovo **svaki servis** može virtuelizovati
- Najšira definicija: **heterogeno polje resursa** se krajnjem korisniku predstavlja kao **nešto homogeno**
- Primer: **elektrosnabdevanje**
  - potrošač ne zna da li struja dolazi iz termo-, hidro- ili nuklearne elektrane
  - proizvodnja, skladištenje i transport su **apstrahovani**

BAZIČNA ARHITEKTURA VIRTUELIZACIJE

- **Virtuelna mašina (VM)** – zamišljeni računar koji u stvarnosti radi na nekom fizičkom računaru
- **Host** (domaćin) – fizički računar sa **host operativnim sistemom**
- **Guest** (gost) – VM sa svojim **guest operativnim sistemom**
- Ključna osobina: guest OS i sav softver na njemu **ne zna** da je na VM-u, a ne na fizičkom računaru

DA LI VM ZNA DA JE VM?

- Ne baš 100%:
  - **korisnički softver** – po pravilu **ne zna**
  - **operativni sistem** – donekle **jeste svestan** promenjenog okruženja
- Nije neophodno, ali se OS često **modifikuje radi boljih performansi**
- Tipičan primer: **Guest Additions** (VirtualBox)

ŠTA OMOGUĆAVA VIRTUELIZACIJU?

- Mehanizam koji upravlja virtuelizacijom ima više imena:
  - **menadžer** / **monitor virtuelnih mašina** (VMM)
  - najčešće: **hipervizor** (eng. *hypervisor*)
- Odakle ime? Kernel OS-a se često zove **supervisor**
  - nešto iznad *super* mora biti **hyper**

TIPOVI HIPERVIZORA

- Dve glavne forme:
  - **Tip 1** – *bare metal* (“goli metal”)
  - **Tip 2** – *hosted* (“ugošćeni”)
- Ova podela **nije iscrpna**

HIPERVIZORI TIPA 1

- Izvršava se **direktno na hardveru**, bez ičega ispod sebe
- Ima neke osobine OS-a, ali **nije klasičan OS**
- **Najbolje performanse** – najmanje slojeva između VM-a i hardvera
- Primeri: **Xen**, **VMware ESXi**

HIPERVIZORI TIPA 2

- Izvršava se kao **proces** host operativnog sistema
- Prednosti:
  - **bezbednost** – proces je dodatno ograničen od strane OS-a
  - lako se koristi na **ličnim računarima** i računarima mešane namene
- Mana: **performanse**
  - zahtev prolazi kroz više slojeva: guest → emulirani hardver → host OS → pravi hardver
  - performanse su generalno **znatno lošije**
- Primer koji vam je *jako* poznat: **VirtualBox**

HIPERVIZORI TIPA... 1.5?

- Gde svrstati **KVM** (Kernel-based Virtual Machine)?
- KVM je **modul Linux kernela** koji Linux efektivno **pretvara u hipervizor tipa 1**
- Aplikacije u **korisničkom prostoru** mogu da zatraže virtuelizaciju **“na metalu”**
- Tipična kombinacija: **QEMU/KVM**
  - Veljkova omiljena kombinacija

PITANJE HARDVERA

- Izvršavanje koda i memoriju je lako **odvojiti** – problem nastaje kod **ulaza/izlaza**
- Hardver se ne može jednostavno **deliti direktno**:
  - deset VM-ova, **jedan disk** – ko piše u superblok?
  - čiji je koji **mrežni paket**?
- Rešenje: hardver se **emulira**

EMULIRANI HARDVER

- Svi uređaji VM-a postoje kao **emulirane softverske apstrakcije**
- Tipičan primer: **QEMU** – emulator sistema
- Posebno je zanimljiv jer je **potpun emulator sa opcionom primenom hipervizora**

POTPUN EMULATOR SA OPCIONOM PRIMENOM HIPERVIZORA

- QEMU može da radi na **dva načina**:
  - emulira **sav hardver**, uključujući i **procesor**
  - emulira **sve osim procesora**, koji se virtuelizuje kroz hipervizor (npr. **KVM**)
- Konfiguraciju virtuelnog računara **sastavljamo pri pokretanju** VM-a

KVM I QEMU

- **KVM** (Kernel-based Virtual Machine) – modul Linux jezgra koji, uz **VT-x/AMD-V**, pretvara Linux u hipervizor
  - virtuelizuje **procesor i memoriju**; gost se izvršava direktno na CPU-u
  - svaka VM je **Linux proces**, svaki vCPU jedna **nit**
- **QEMU** (Quick Emulator) – program u korisničkom prostoru
  - emulira **uređaje**: disk, mrežu, BIOS/UEFI, grafiku
  - bez KVM-a emulira i procesor (sporo, ali radi i za **druge arhitekture**)
- Zajedno: **KVM = brzina**, **QEMU = kompletan računar**

O ILUSTRACIJI

- Svaka VM emulira *nešto*
- Primer je izabran tako da se lako **testira kod kuće**
  - potreban je samo noviji **Linux kao host OS**
- Komponente:
  - **KVM** – hipervizor
  - **QEMU** – emulator
  - **libvirt** – API sloj za upravljanje VM-ovima

LIBVIRT

- Ne donosi **nove sposobnosti**
- Pruža **uniforman način** da se opiše i upravlja ponašanjem VM-ova
- Nas najviše zanima njegov **XML jezik** za opis konfiguracije VM-a

DOMENI

- Sve što emulira, libvirt zove **domen**

```
<domain type='kvm' id='1'>
  <name>MyGuest</name>
  <uuid>4dea22b3-1d52-d8f3-2516-782e98ab3fa0</uuid>
  <genid>43dc0cf8-809b-4adb-9bea-a9abb5f3d90e</genid>
  <title>A short description - title - of the domain</title>
  <description>Some human readable description</description>
  <metadata>
    <app1:foo xmlns:app1="http://app1.org/app1/">..</app1:foo>
    <app2:bar xmlns:app2="http://app1.org/app2/">..</app2:bar>
  </metadata>
```

(Primeri iz https://libvirt.org/formatdomain.html)

EMULACIJA PROCESORA

- Za CC emulacija procesora **nema veliku korist**
- Korisna je za: razvoj za **egzotične platforme**, eksperimente, **softversku arheologiju**
- Koraci:
  1. tip domena se promeni tako da **ne koristi hipervizor** (npr. `type='qemu'`)
  2. podesi se **željeni oblik procesora**

ŽELJENI OBLIK PROCESORA

```
<cpu match='exact'>
  <model fallback='allow'>core2duo</model>
  <vendor>Intel</vendor>
  <topology sockets='1' dies='1' cores='2' threads='1'/>
  <cache level='3' mode='emulate'/>
  <maxphysaddr mode='emulate' bits='42'/>
  <feature policy='disable' name='lahf_lm'/>
</cpu>
```

VIRTUELIZACIJA KROZ HIPERVIZOR I PROCESOR

- Ne želimo emulaciju, već da VM koristi **pravi procesor** host mašine
- VM tada treba da vidi **iste mogućnosti** (instrukcije, ekstenzije) kao fizički procesor
- Srećom, za to postoji **prečica**

PREČICA

```
1 <cpu mode="host-model" check="partial"/>
```

- **Kopira model fizičkog procesora** na virtuelizovani (ali **ne emulirani**) procesor

ALOKACIJA PROCESORA

- VM-ovi **ne moraju** imati iste resurse
- Upravo ono što CC obećava: resurse **agregiramo na jednom mestu**, a **particionišemo na drugom**
- VM dobija **virtuelna jezgra (vCPU)**
  - biramo na kojim **fizičkim jezgrima** se ona izvršavaju

PRIMER ALOKACIJE VIRTUELNIH PROCESORA

```
<vcpu placement='static' cpuset="1-4,^3,6" current="1">2</vcpu>
<vcpus>
  <vcpu id='0' enabled='yes' hotpluggable='no' order='1'/>
  <vcpu id='1' enabled='no' hotpluggable='yes'/>
</vcpus>
```

PRIMER ALOKACIJE VIRTUELNIH PROCESORA

- **Statički** alocira **2 vCPU-a** (`current="1"` – na početku je aktivan jedan)
- Izvršavaju se na fizičkim jezgrima **1–4 bez 3, plus 6** (`cpuset="1–4,^3,6"`)
- **vcpus** podešava svaki vCPU posebno: da li je uključen na startu (enabled) i da li se može dodati/ukloniti u radu (hotpluggable)

EMULACIJA BLOKOVSKIH UREĐAJA

- Procesori nisu problem – ozbiljnije pitanje je **disk**
- Za instalaciju guest OS-a potreban je **neki disk**
- **libvirt** ga specificira, **QEMU** ga emulira – onakav kakav izaberete

EMULACIJA BLOKOVSKIH UREĐAJA

```
1     <disk type="file" device="disk">
2       <driver name="qemu" type="qcow2"/>
3       <source file="/putanja/do/fajla/gde/je/disk"/>
4       <target dev="vda" bus="virtio"/>
5       <address type="pci" domain="0x0000" bus="0x04" slot="0x00" function="
6     </disk>
```

EMULACIJA BLOKOVSKIH UREĐAJA

- Emulirani disk je na hostu **običan fajl**
- Format **qcow2** (QEMU Copy-On-Write) – optimizovan baš za ovu namenu
  - odlična **copy-on-write** semantika
  - fajl **raste tek kada je prostor zaista potreban** → manje bačenog prostora

PROBLEM SA EMULACIJOM HARDVERA

- Emulacija hardvera je **spora**
  - naročito kod **I/O operacija**
  - još gore kod **interaktivne** upotrebe VM-a
- Dva moguća rešenja:
  - **Paravirtuelizacija**
  - **Prespoj na nivou uređaja**

PARAVIRTUELIZACIJA

- **Paravirtuelizacija (PV)** – virtuelizacija **nije transparentna** za guest OS
  - OS je **modifikovan** da bi se lakše virtuelizovao
- Dve forme:
  - **potpuna** paravirtuelizacija
  - **parcijalna** paravirtuelizacija

POTPUNA PARAVIRTUELIZACIJA

- Ceo OS je modifikovan **na najdubljem nivou** za rad u virtuelizovanom okruženju
- Okruženje je **potpuno netransparentno**
  - umesto komunikacije sa hardverom, OS poziva **hipervizor** (*hypercalls*)
- Pokretanje "zna" da nema prave mašine
  - **nema inicijalizacije hardvera** – kernel se pokreće odmah

POTPUNA PARAVIRTUELIZACIJA

- Bila je **izuzetno brza** (komparativno)
  - umesto teške virtuelizacije osobina procesora, sistem **samo kaže hipervizoru šta želi**
- I dalje je **transparentna za korisnički softver**
  - deo OS-a koji pruža interfejs aplikacijama **se ne menja**

SMANJENJE VAŽNOSTI PARAVIRTUELIZACIJE

- Nekad je PV bila **jedini praktičan način** za prihvatljive performanse
- Danas je PV i dalje brza, ali je **HVM** (*Hardware Virtual Machine*) **dostigao i prestigao**
- Kako? **Hardverska podrška u procesoru** (Intel **VT-x**, **AMD-V**)
  - nepromenjen OS dostiže performanse **bliske "golom metalu"**
- PV je i dalje jedina praktična opcija **bez hardverske podrške**

PARCIJALNA PARAVIRTUELIZACIJA

- Ipak, PV se i dalje koristi – u **parcijalnoj formi**
- HVM je brz, ali ostaje staro usko grlo: **simulirani hardver**
- Gubitak vremena: sistem se pretvara da priča sa **fiktivnim diskom**
  - umesto da zahtev pošalje **direktno hipervizoru**

PARCIJALNA PARAVIRTUELIZACIJA

- Poznata i kao **PV on HVM** (npr. kod **Amazon**-a)
- Ideja: promeniti samo **deo OS-a** – onaj koji se lako menja: **drajvere**
  - PV nam treba baš za **I/O**, a I/O se radi upravo **kroz drajvere**
- Drajveri za virtuelni hardver **direktno pozivaju hipervizor** umesto spore emulacije
- Rezultat: **pune performanse** uz **maksimalnu fleksibilnost**

PRIMER DISKA (OPET)

```
1     <disk type="file" device="disk">
2       <driver name="qemu" type="qcow2"/>
3       <source file="/putanja/do/fajla/gde/je/disk"/>
4       <target dev="vda" bus="virtio"/>
5       <address type="pci" domain="0x0000" bus="0x04" slot="0x00" function="
6     </disk>
```

PRIMER DISKA (OPET)

- Ovde je već uključena **parcijalna paravirtuelizacija**
- QEMU/KVM radi **HVM**, uz podsistem za parcijalnu PV
- Taj podsistem se zove **virtio** (`bus="virtio"`)

PRESPOJ NA NIVOU UREĐAJA

- HVM + parcijalna PV radi samo za hardver koji se **i dalje virtuelizuje**
- Neki hardver je **teško virtuelizovati** – klasičan primer je **GPU**
- Rešenje (*device passthrough*): hardverska podrška (**VT-d** / **IOMMU**) usmerava svu komunikaciju sa fizičkim uređajem **na jednu VM**

PRESPOJ NA NIVOU UREĐAJA

- Kao da se pravi hardver **“utakne” u VM**
- Može se raditi na nivou:
  - **PCI kartice** – najčešće GPU, ali može i nešto drugo
  - **USB uređaja** – tzv. *USB redirekcija*
- Omogućava npr. CC instance **sa grafičkim karticama**

MODERNI STANDARD

- Današnji standard, sa performansama **jedva razlučivim od rada bez virtuelizacije**:
  - **HVM** – za glavninu posla
  - **parcijalna PV** – za emulirani (većinom I/O) hardver
  - **prespoj na nivou uređaja** – za GPU, ako je potreban
- Dostupan na praktično **svakom OS-u**, i na serverima i na ličnim računarima

KONTEJNERI I VIRTUELNE MAŠINE

- Virtuelizacija je **neizbežna i korisna**, ali...
- Često se VM koristi da bi se u OS-u X pokretale **instance istog OS-a X**
  - samo da bi aplikacije bile **zatvorene u svoja okruženja**
- Sve sposobnosti VM-a (simulirani hardver, drugi OS-ovi, transparentnost) su tada **bačene**

ZAŠTO EMULIRATI LINUX U LINUX-U?

- Glavni razlozi da se u produkciji pokreće VM **identična domaćinu**:
  - **Portabilnost**
  - **Ponovljivost**
  - **Izolacija**

PORTABILNOST

- Aplikaciji treba mnogo **potpornog softvera**: biblioteke, alati, serveri...
- Zato je **teško poslati** nekome aplikaciju
  - korisnički sistemi: složena **instalacija** ili složen **menadžer paketa**
- Kod servera je još gore – javlja se **konflikt zahteva** između aplikacija

KONFLIKT ZAHTEVA

- Na istoj mašini **paralelno** radi mnogo serverskih aplikacija
- Aplikacija A traži **jednu verziju biblioteke**, aplikacija B **drugu** – čija je “starija”?
- Čak i kad se to reši (virtuelna okruženja, pakovanje zavisnosti), ostaju **različite verzije servera**
  - npr. dve Java aplikacije, svaka traži **drugačiju verziju `Tomcat`-a**

KONFLIKT ZAHTEVA

- Ostaju i **interfejsi koji se ne mogu replicirati**
  - dva web servera – ko dobija portove **80 i 443**?
- Pokretanje aplikacije postaje **dug i mučan proces** pun konfiguracije i kompromisa
  - teško se **automatizuje**
- CC zahteva **“samo pokreni”** → portabilnost kroz virtuelizaciju je **neophodna**

PONOVLJIVOST

- U produkciji ne sme da se desi da **sitna promena** konfiguracije:
  - **pokvari** softver, ili još gore
  - učini da radi **drugačije**
- Dobro poznata boljka: **“Ali radi na mojoj mašini”**
- Potreban je **recept** (idealno automatizovan) koji proizvodi **tačno** okruženje za aplikaciju

IZOLACIJA

- Sprečavamo **slučajne**, ali i **namerne** konflikte
- Napad na **jedan** serverski proces → ugroženi su **svi** procesi na sistemu
- Separacija procesa postoji, ali je (podrazumevano) **nesavršena**
  - Zašto?

ZAŠTO JE SEPARACIJA PROCESA NESAVRŠENA?

- Dobro je odvojena samo **memorija** – skoro **sve ostalo je zajedničko**
- **Zajednički fajl sistem**
  - svi vide **isto stablo direktorijuma**, štite ga samo **dozvole**
  - mnogo toga je čitljivo svima: konfiguracije, logovi, deljeni `/tmp`
- **Sve je vidljivo**
  - `ps` prikazuje **sve procese** (i njihove argumente)
  - **ista mreža** – servisi na `localhost` često **bez lozinke**

ZAŠTO JE SEPARACIJA PROCESA NESAVRŠENA?

- `root` **je svemoćan**
  - mnogi servisi rade kao root → proboj u servis = **preuzet sistem**
- **Jedan zajednički kernel**
  - **stotine sistemskih poziva** → velika **površina napada**
  - jedna greška u kernelu → **eskalacija privilegija** → pada sve
- **Nema ograničenja resursa**
  - jedan proces može da potroši **sav CPU, memoriju ili disk** (npr. *fork bomb*)

IZOLACIJA

- Izolacija između VM-ova je **znatno bolja**
- “Bekstvo” iz VM-a je moguće, ali je **za red veličine teže**
  - **površina napada** je mnogo bolje definisana → lakša odbrana
- Potencijalno maliciozan sadržaj ostaje **“zarobljen”** → bezbedniji ceo sistem

MOTIVACIJA ZA KONTEJNERE

- **Ne treba** nam poseban hardver, egzotičan CPU ni novi OS → VM **troši resurse uzalud**
- Da li postoji **lakša virtuelizacija**?
  - samo **odvojena korisnička okruženja** (*userspace*)
  - koja **dele isti kernel**
- Ako da – kako bi to radilo?

OPŠTA IDEJA

- **Virtuelizacija na nivou operativnog sistema**
- Ne pretvaramo se da imamo **novi računar**
- Umesto toga, OS se **pretvara da je više različitih instalacija**
  - svaka grupa procesa vidi **svoju instalaciju** → izolacija

OPŠTA IDEJA

- Moguće je jer procesi **u potpunosti zavise od OS-a** da saznaju šta se dešava
- Ako OS “laže” na sistemskim pozivima – program to **ne može da otkrije**
- Izazov: OS mora da drži **konzistentne skupove laži** i da govori **prave laži pravom softveru**
- Zato se ovo radi kombinacijom **podrške u kernelu** i **alata u korisničkom prostoru**

PRETEČA – CHROOT

OGRANIČENJA

- `chroot` je **stara ideja** sa **slabom bezbednošću**
  - štiti **samo fajlove**, ništa drugo
- Ipak, iz nje direktno nastaje **BSD `jail`** (1999)
  - sistem se deli na **“zatvore”** koji su rigorozno odvojeni
  - ima i **bezbednosne komponente** → vrlo blizu **modernim kontejnerima**

TEHNOLOGIJA IZA MODERNIH KONTEJNERA

- Moderni kontejneri se oslanjaju na mehanizam Linux kernela: **sistemski prostori imena (SPI)** (eng. *namespaces*)
- Ideja: kernel deli resurse sistema na **particije** i svakom procesu **prikazuje samo jednu**
- Dva procesa mogu da vide **dva potpuno disjunktna skupa resursa**

ŠTA SE MOŽE ODVOJITI U SPI

Linux poseduje **osam tipova** SPI:

1. `Cgroup` – poseban skup **kontrolnih grupa** (merenje i ograničavanje resursa)
2. `IPC` – posebni resursi za **komunikaciju između procesa**
3. `Network` – sopstveni **mrežni stek**: interfejsi, rutiranje, firewall
4. `Mount` – sopstveni **skup montiranih fajl sistema**

ŠTA SE MOŽE ODVOJITI U SPI

5. `PID` – sopstvena **tabela procesa**, bez procesa drugih SPI
6. `Time` – sopstvena podešavanja **sistemskog vremena**
7. `User` – sopstveni **korisnici** (uključujući "svog" `root`-a) i tumačenje **dozvola**
8. `UTS` – sopstveni `hostname`

KAKO ODAVDE DO KONTEJNERA

- **SPI** (rigorozno isečeni resursi) + **`chroot`** (zarobljen fajl sistem)
  - = nešto što **neodoljivo podseća na kontejner**
- Kontejner se može napraviti **jednom komandom** iz paketa **`util-linux`**: `unshare`

UNSHARE

```
1 unshare -r -i -p -f --kill-child -R /home/veljko/tmp2 ./tst
```

OBJAŠNJENJE

- `-r` – korisnik se unutar kontejnera mapira na `root`-a (implicitno pravi i novi **User** SPI)
- `-i` – novi, odvojen **IPC** SPI
- `-p` – novi, odvojen **PID** SPI
- `-f` – pokreće se **novi proces** (*fork*) sa svim navedenim podešavanjima
- `--kill-child` – (grozomorno ime) kada se `unshare` završi, **gase se i svi potomci**
- `-R` – novi **koreni direktorijum** za novi proces (kao `chroot`)

OBJAŠNJENJE

- `./tst` – trivijalni C program koji radi isto što i `ls /`
- Demonstrira da je za novi proces korenski direktorijum potpuno odvojen od ostatka sistema

tst.c

```c
#include <dirent.h>
#include <stdio.h>

int main() {
  DIR *d;
  struct dirent *dd;
  d = opendir("/");
  if (d) {
    while ((dd = readdir(d)) != NULL) {
      printf("%s\n", dd->d_name);
    }
    closedir(d);
  }else{
    fprintf(stderr, "Ne mogu da otvorim korenski direktorijum.\n");
  }
  return 0;
}
```

REZULTAT

```
.
..
chroot_works
tst.c
tst
usr
lib
lib64
```

REZULTAT

- Izlistan je sadržaj **`tmp2`**, kao da je to **`/`**
- Ograničenje: program mora imati pristup **`glibc`** (dinamički je linkovan)
- Zato su direktorijumi sa bibliotekama (`usr`, `lib`, `lib64`) **povezani u `tmp2`**
- U praksi: alati sa **malo zavisnosti** ili **statički kompajlirani** programi

ŠTA NAM OVO GOVORI?

- Ovo je **dovoljno** da počnete razvoj svoje verzije **Docker**-a
- Docker radi **mnogo više**, na mnogo složeniji način
- Ali **centralna ideja je ista**
  - umesto komande koristi se **sistemski poziv** istog imena (`unshare`)

ŠTA JE JOŠ POTREBNO?

- Ako sve ovo radi **obična komanda**, šta onda radi **Docker**?
- Mnogo toga, ali jedna od glavnih stvari je ono što je u primeru **preskočeno**
- Povezivanje svih biblioteka (pomoću `mount -o bind`) **nije trivijalno**
- Veliki izazov: **pripremiti okruženje** za kontejner

ŠTA JE JOŠ POTREBNO?

- Okruženje za VM je složeno, ali liči na **instalaciju OS-a** – to **znamo** da radimo
- Kako “instalirati” nešto što **malo jeste, a malo nije** poseban OS?
- U svetu kontejnera: **image** (slika) – neka vrsta **“dehidriranog” kontejnera**
- Više o tome **kasnije**

KRAJ

Hvala na pažnji!